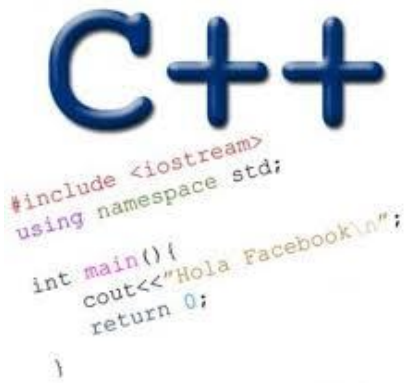
STACKS AND QUEUES

Problem Solving with Computers-II

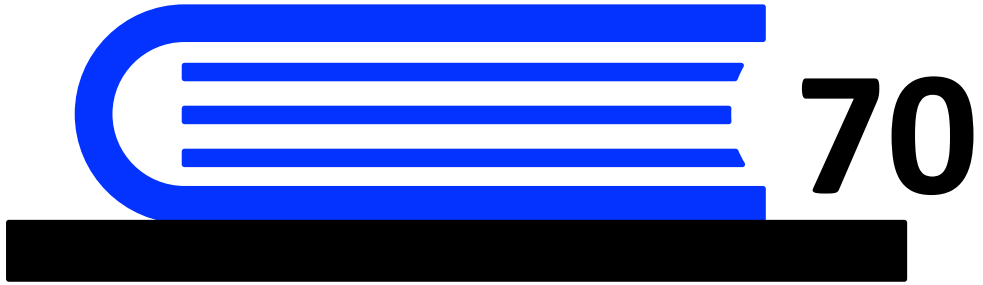


stack<int> s

Empty stack 

Operations: **push()** **pop()** **top()**

stack<int> s
s.push(70)



Operations: **push()** **pop()** **top()**

stack<int> s

s.push(70)

s.push(50)



Operations: **push()** **pop()** **top()**

stack<int> s

s.push(70)

s.push(50)

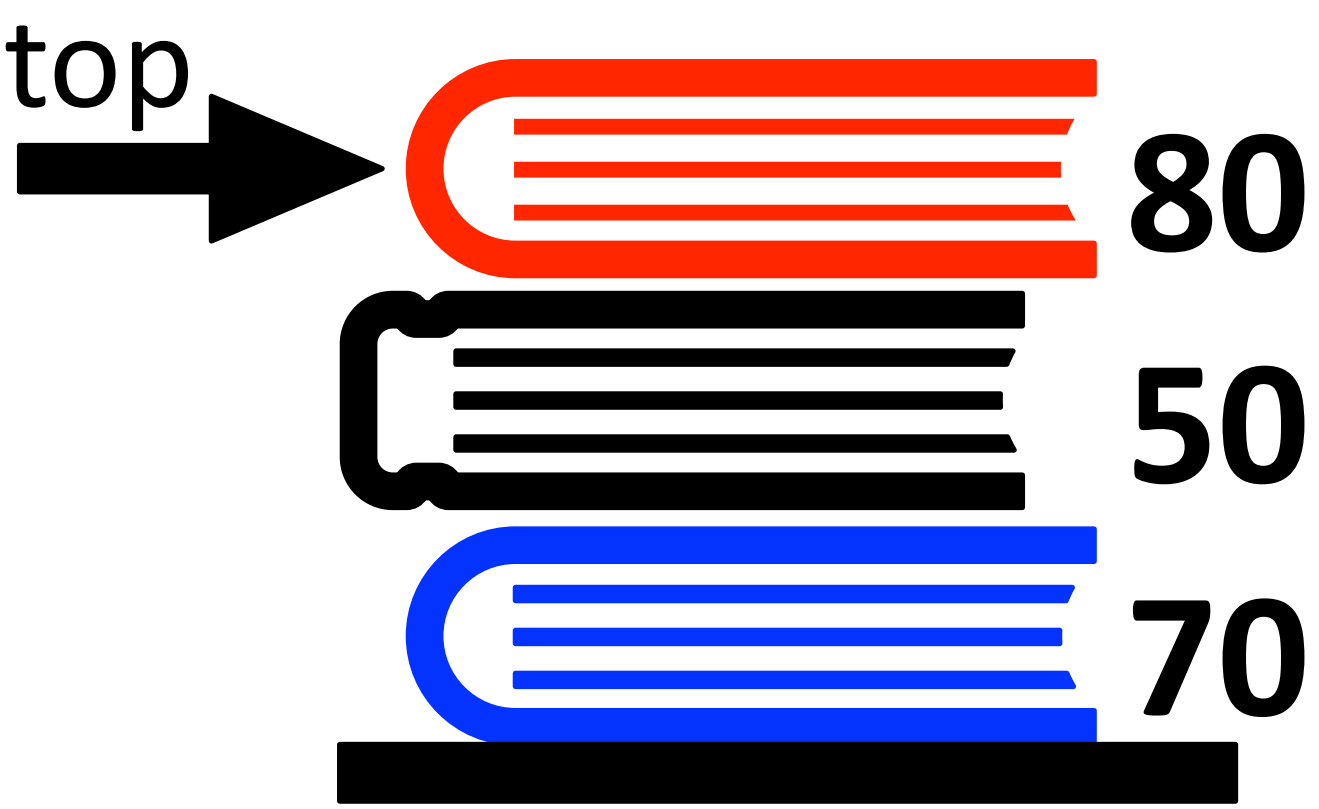
s.push(80)



Operations: **push()** **pop()** **top()**

```
stack<int> s
s.push(70)
s.push(50)
s.push(80)
```

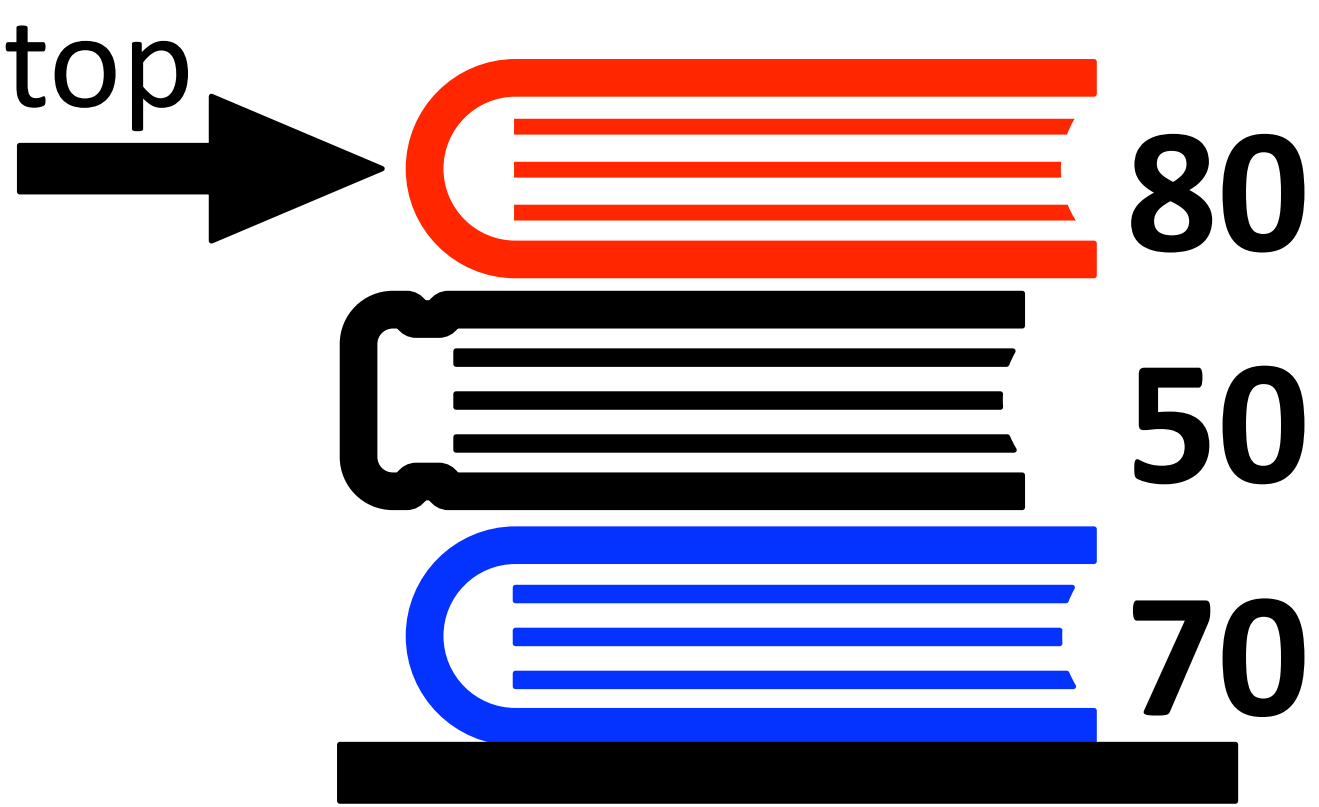
s.top() returns 80



Operations: push() pop() **top()**

```
stack<int> s
s.push(70)
s.push(50)
s.push(80)

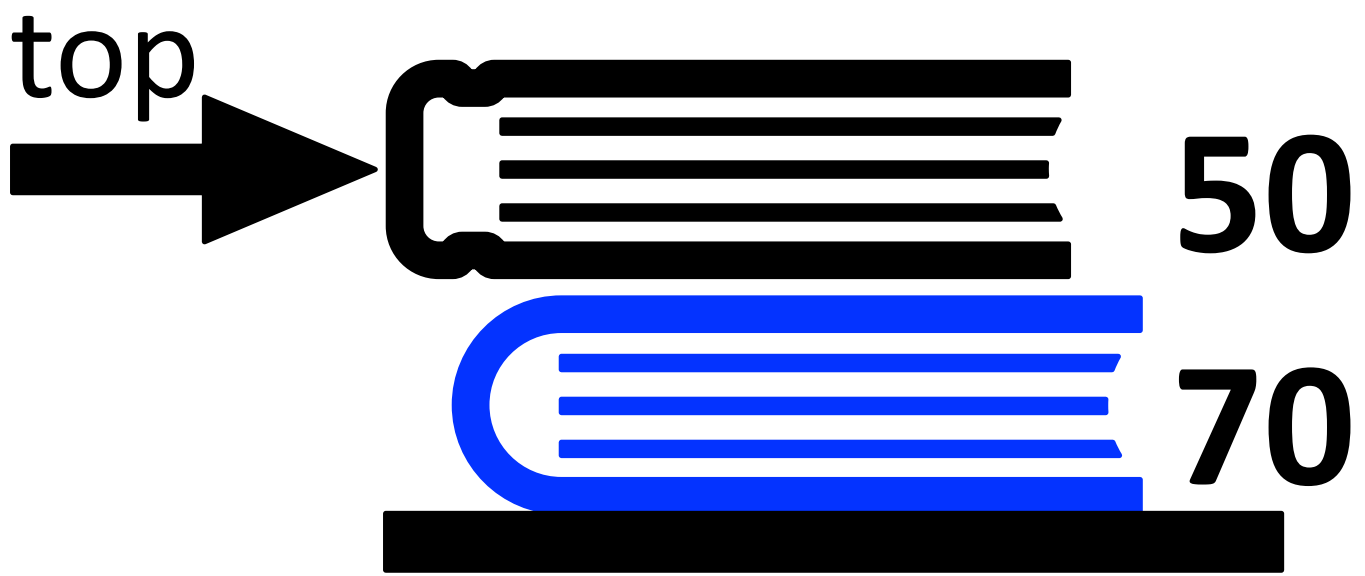
s.top()
```



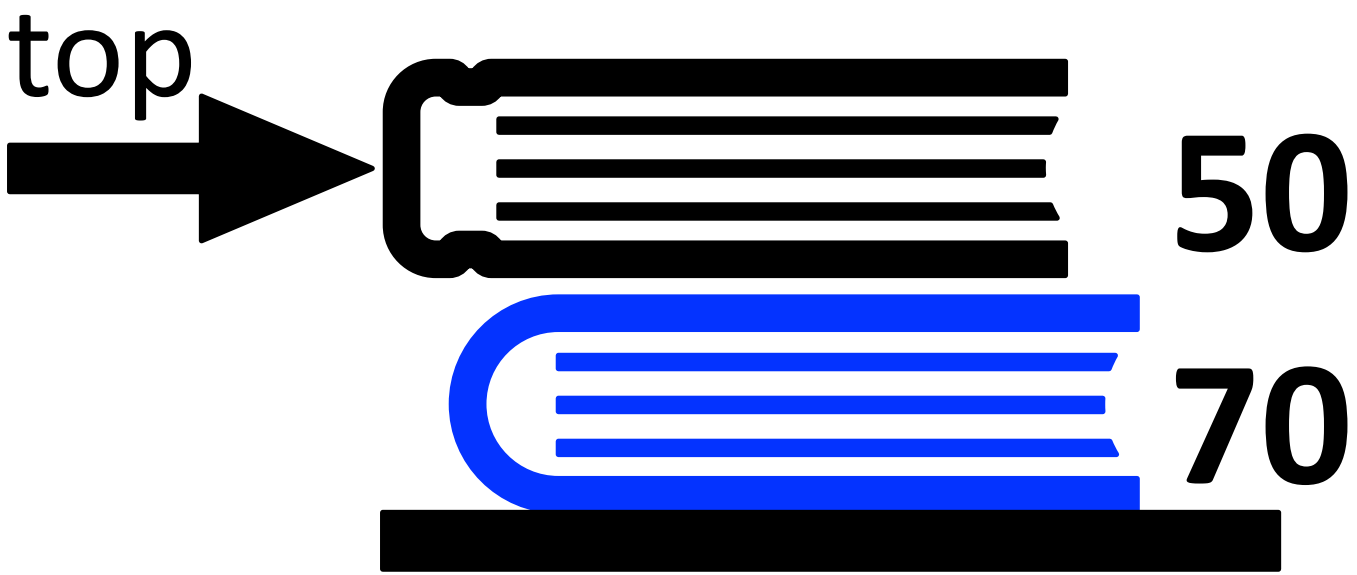
s.pop() removes value that was pushed in *last*

```
stack<int> s
s.push(70)
s.push(50)
s.push(80)

s.top()
```



s.pop() removes value that was pushed in *last*



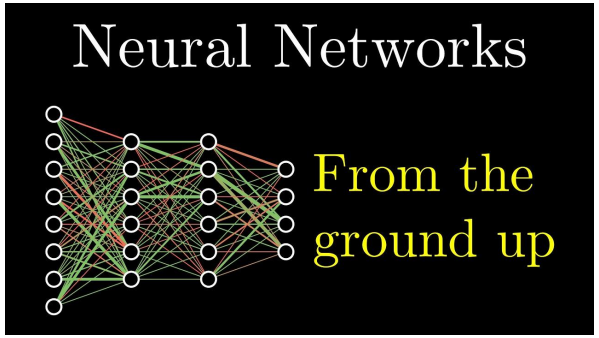
The Last value In is the First value Out (LIFO)

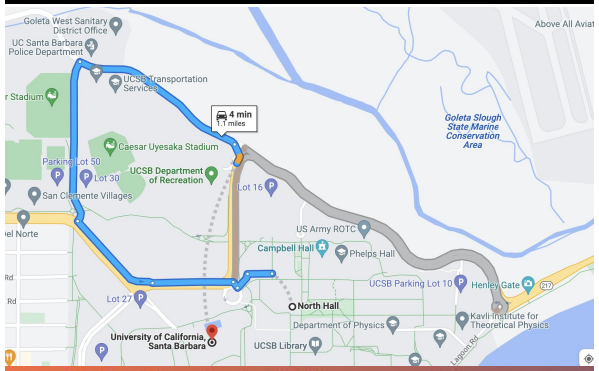
Why implement a stack at all?
After all a stack is a vector or linked list with a **reduced set of operations**

A stack is useful for keeping track of history information where computation only depends on the most recent information !!

Stack has only three operations: **push()** **pop()** **top()**

ADT	Algorithm	Real-World Application
Stack	Depth-First Search	Machine Learning (**PA03**: Training and Prediction in Neural Nets)
Queue	Breadth-First Search	
Priority Queue	Dijkstra's Algorithm	GPS Navigation (Shortest path)
	Huffman Coding	Data Compression (ZIP, JPEG, MP3)
Your choice!	You design!	Querying a movie dataset **(PA02)**

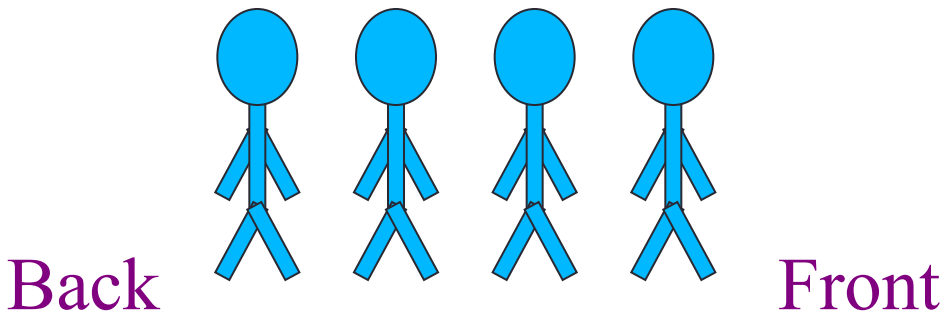






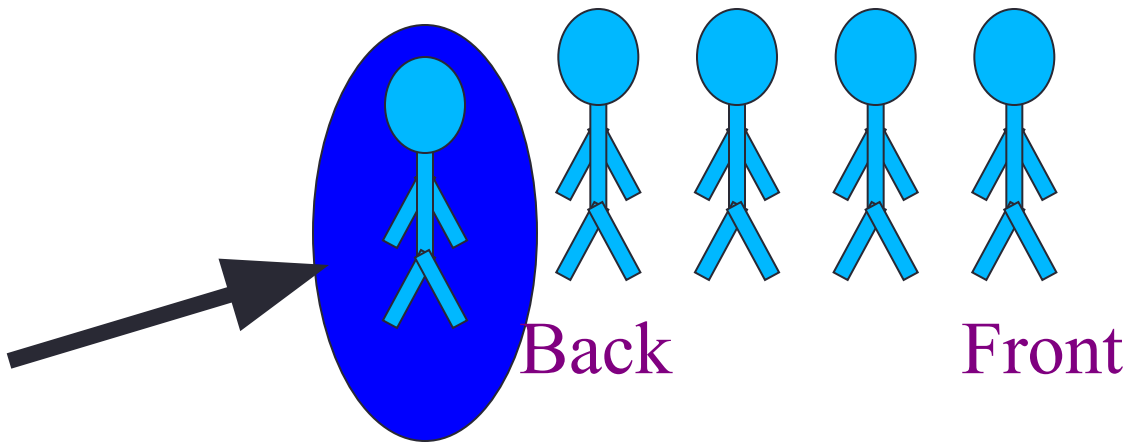
Queue: First come First Serve

- A queue is like a queue of people waiting to be serviced
- The queue has a **front** and a **back**.



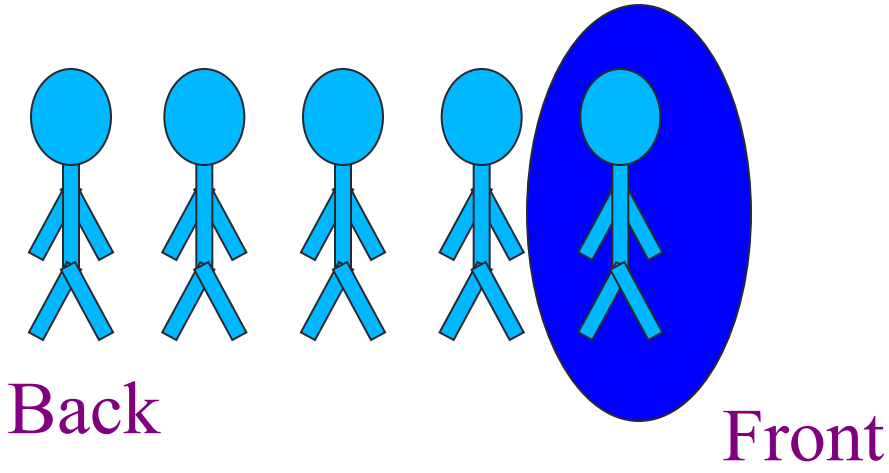
Queue Operations: push, pop, front, back

New people must enter the queue at the back. The C++ queue class calls this a **push** operation.



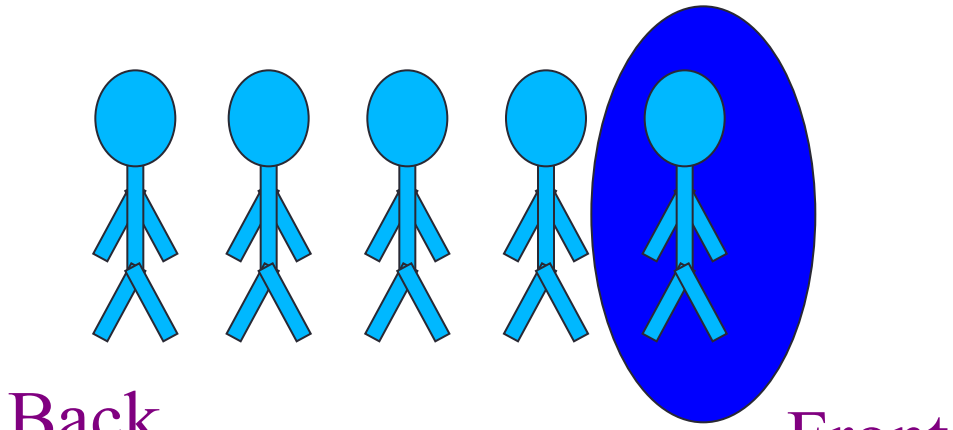
Queue Operations: push, pop, front, back

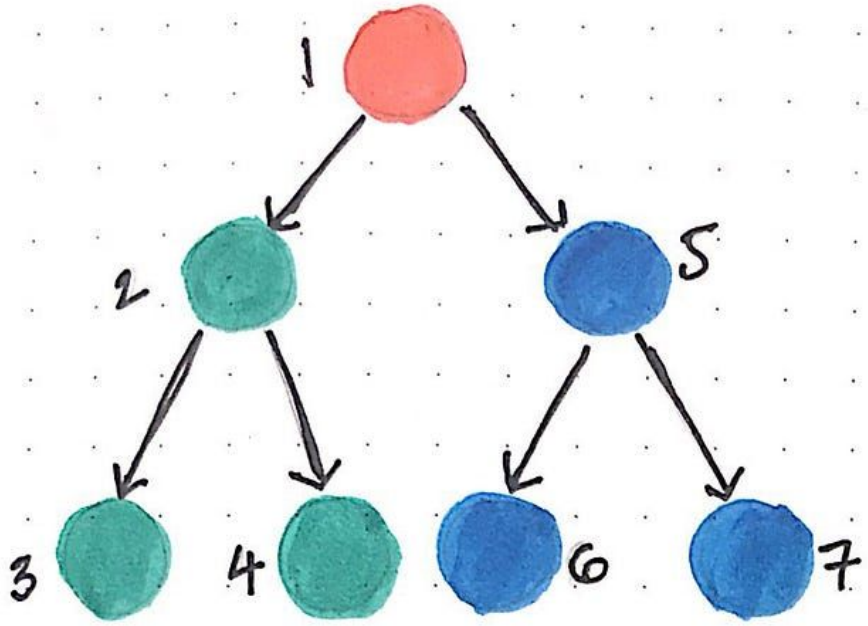
- To check the item in the front of the queue, use **front()**
- To check the item at the back of the queue, use **back()**
- When an item is taken from the queue, it always comes from the front.
- To delete an element from the front of the queue, use **pop()**



Queue Operations: empty(), push, pop, front, back: **O(1)**

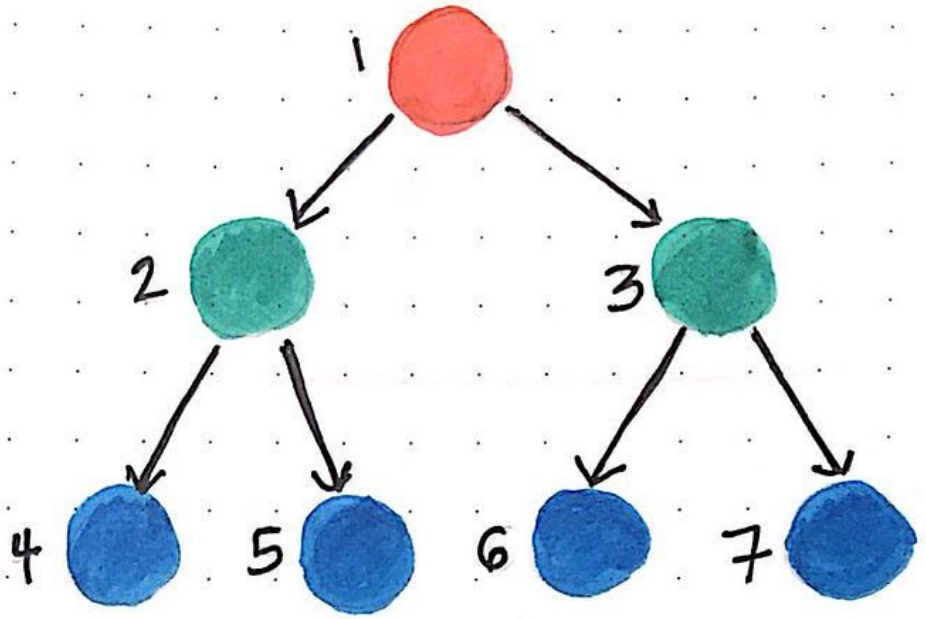
```
std::queue<int> q;
q.empty(); //true
q.push(1);
// push 2, 3, 4, 5
q.front();
q.back();
q.pop();
```





Depth-first search

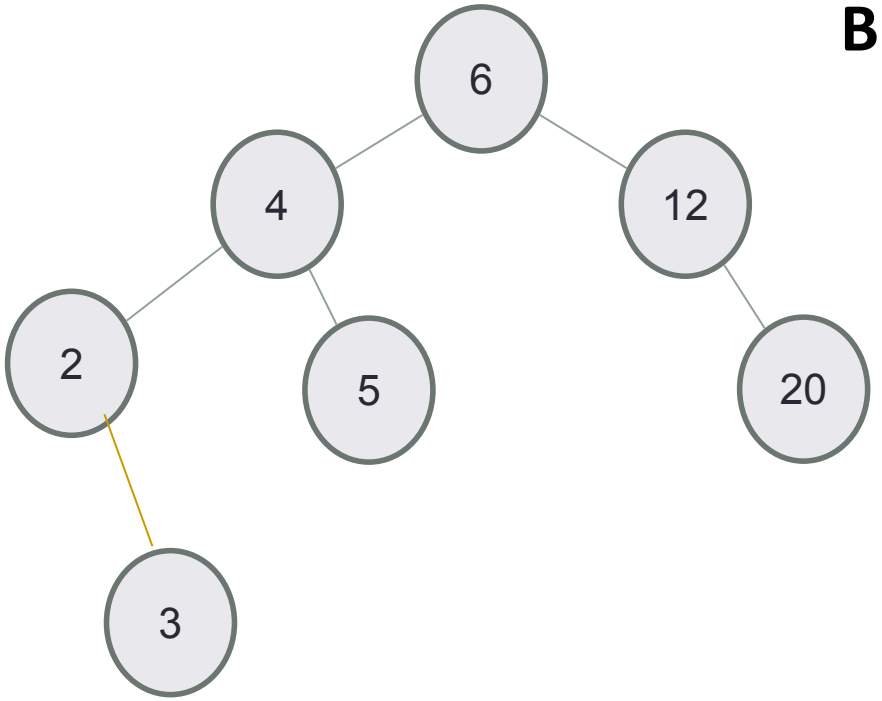
- Traverse through left subtree(s) first, then traverse through the right subtree(s).



Breadth-first search

- Traverse through one level of children nodes, then traverse through the level of grandchildren nodes (and so on...).

Breadth-first traversal/search



BFS Algo:

- Create an empty **queue**.
- Insert the **root** into the **queue**.
- While queue is not empty,
  - Print the key in the front of the queue
  - Push all the children of the node into the queue.
  - Pop the front node from the queue

```
mystery(root: pointer to root of a binary tree):
    best = -1
    frontier = empty queue
    if root is not null:
        frontier.push((root, 0))
    while frontier is not empty:
        (node, level) = frontier.front()
        frontier.pop()
        best = level
        if node.left is not null:
            frontier.push((node.left, level + 1))
        if node.right is not null:
            frontier.push((node.right, level + 1))
    return best
```

`mystery()` is a variation of which traversal?

A) Preorder

B) Inorder

C) Postorder

D) Breadth first

```
mystery(root: pointer to root of a binary tree):
    best = -1
    frontier = empty queue
    if root is not null:
        frontier.push((root, 0))
    while frontier is not empty:
        (node, level) = frontier.front()
        frontier.pop()
        best = level
        if node.left is not null:
            frontier.push((node.left, level + 1))
        if node.right is not null:
            frontier.push((node.right, level + 1))
    return best
```

What does `mystery(root)` compute?

A) The number of nodes in the tree

B) The height of the tree

C) The number of levels in the tree, counting the root's level as 1

D) The depth of the rightmost leaf